OPEN HACKATHONS
powered by OpenACC

STUDENT GUIDE
2026 EDITION

SINICA × NCHC

CUDA-Q Agentic Coding Bootcamp

Student Hands-on Guide

DATE & TIME
August 7, 2026
09:30–16:30 (Taiwan)

CLASSROOM
NCHC H200 VM
CUDA-Q + Agentic AI

ORGANIZED IN COLLABORATION WITH

中央研究院 ACADEMIA SINICA
中央研究院 關鍵議題研究中心 Research Center for Critical Issues | Academia Sinica
NATIONAL INSTITUTES OF APPLIED RESEARCH National Center for High-performance Computing
OPEN HACKATHONS powered by OpenACC
NVIDIA

BOOTCAMP AT A GLANCE

Event Information

A one-day hands-on course organized by the Research Center for Critical Issues (RCCI), Academia Sinica; the National Center for High-performance Computing (NCHC); NVIDIA; and Open Hackathons. Participants use an NCHC H200 environment for CUDA-Q fundamentals, NVIDIA Ising, NCHC RAP, and agentic quantum coding exercises.

DATE & TIME

Friday, August 7, 2026
09:30–16:30 (Taiwan Time)

FORMAT

In-Person
Hands-on Bootcamp

VENUE

Academia Sinica Southern Campus
RCCI Building, 1F Lecture Hall
No. 100, Sec. 1, Gui-Ren 13th Rd., Guiren Dist., Tainan City

COURSE AI SERVICE

NCHC RAP • Agentic coding model service

OFFICIAL LINKS

Event registration • Open Hackathons
Event information and agenda • nqobu/nvidia/20260807
CUDA-Q Agentic Coding course • GitHub main
Course AI service • Taiwan AI RAP

This guide contains no activity API key, Jupyter token, reference solution, or instructor image-packaging information. IP addresses are examples only.

Agenda 議程

**All times are in Taiwan Standard Time (台灣標準時間)*

Friday, August 7, 2026 // 09:30 AM - 04:30 PM (In-Person) (實體活動)

TIME 時間	SESSION 課程內容
09:30 AM – 10:00 AM 上午 09:30 – 10:00	**Registration & Setup** 報到與環境設定
10:00 AM – 10:15 AM 上午 10:00 – 10:15	**Welcome & Introduction (NCHC + NVIDIA)** 歡迎致詞與課程介紹（NCHC + NVIDIA）
10:15 AM – 10:45 AM 上午 10:15 – 10:45	**NVIDIA Quantum Platform Overview: CUDA-Q & NVIDIA Ising** NVIDIA 量子運算平台總覽：CUDA-Q 與 NVIDIA Ising
10:45 AM – 11:15 AM 上午 10:45 – 11:15	**CUDA-Q Environment Setup** CUDA-Q 開發環境設定
11:15 AM – 12:00 PM 上午 11:15 – 下午 12:00	**Basic CUDA-Q Hands-on & NVIDIA Ising Calibration Playground** 基礎 CUDA-Q 實作與 NVIDIA Ising 校準體驗
12:00 PM – 01:00 PM 下午 12:00 – 下午 01:00	**Lunch Break** 午餐時間
01:00 PM – 02:00 PM 下午 01:00 – 下午 02:00	**Superconducting QPU Introduction & Agentic Calibration** 超導量子處理器介紹與 Agentic 校準
02:00 PM – 02:30 PM 下午 02:00 – 下午 02:30	**NCHC RAP Service Briefing & Environment Setup** NCHC RAP 服務介紹與環境設定
02:30 PM – 03:15 PM 下午 02:30 – 下午 03:15	**Advanced CUDA-Q Hands-on with RAP** 進階 CUDA-Q 實作（結合 RAP 服務）
03:15 PM – 04:00 PM 下午 03:15 – 下午 04:00	**Bring-Your-Own-Code with CUDA-Q Agentic Coding** 自備程式碼實作：CUDA-Q Agentic Coding
04:00 PM – 04:20 PM 下午 04:00 – 下午 04:20	**Q&A & Open Discussion** 問答與綜合討論

STUDENT GUIDE

Contents

All links are clickable • Every step begins on a new page

What you will use

NCHC has prepared the following for you:

- Ubuntu GPU VM
- NVIDIA H200 and driver
- Docker and NVIDIA Container Toolkit
- CUDA-Q 0.15.0 course container
- JupyterLab, Jupyter AI, and OpenCode
- The `cudaq-agentic-coding` course repository (student environments do not contain reference solutions)

You do not need to install CUDA, build a Docker image, or sign in to GitHub.

The overall data flow is:

```
Your browser
    │ Direct HTTP connection (SSH tunnel is the fallback)
    ▼
NCHC student VM (0.0.0.0:8888, protected by a Jupyter token)
    │ Docker + NVIDIA runtime
    ▼
CUDA-Q course container
    ├─ JupyterLab / Jupyter AI
    ├─ OpenCode + NCHC RAP
    └─ /workspace/cudaq-agentic-coding
               │
               └─ Mounted from /home/ubuntu/cudaq-agentic-coding
```

Step 1: Sign in to your assigned VM with `bootcamp0807.pem`

NCHC will prepare about 40 VMs. Each student will receive:

- The `bootcamp0807.pem` SSH private key
- An assigned VM IP address
- The fixed login account `ubuntu`

The examples below use `140.110.109.117`. Replace it with the IP address assigned by the instructor. Do not sign in to another student's VM.

macOS

1. Put `bootcamp0807.pem` in `Downloads`.
2. Open Terminal and restrict the private key permissions:

```
chmod 600 ~/Downloads/bootcamp0807.pem
```

1. Sign in to the assigned VM:

```
ssh -p 3322 -i ~/Downloads/bootcamp0807.pem ubuntu@140.110.109.117
```

Windows 10/11 PowerShell

Windows 10/11 normally includes the OpenSSH Client. Open PowerShell:

```
ssh -p 3322 -i "$HOME\Downloads\bootcamp0807.pem" ubuntu@140.110.109.117
```

If you see `ssh is not recognized`, go to:

```
Settings → Apps → Optional Features → Add a feature → OpenSSH Client
```

After installation, reopen PowerShell.

First connection

On the first connection, you may see:

```
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

After confirming that the IP address is the VM assigned by the instructor, enter:

```
yes
```

After a successful login, the prompt should look similar to:

```
ubuntu@vm206-2:~$
```

Step 2: Activate, validate, and start the course environment

```
cd ~/cudaq-course-kit
./activate-student-course.sh
```

The script will:

1. Create your `course.env` from the activity credential bundle in the VM image.
2. Generate a unique Jupyter token for this VM.
3. Display a notice explaining that the activity keys expire after the event and that you should use personal keys afterward.
4. Validate the H200, Docker, CUDA-Q, and course notebooks.
5. Start token-protected JupyterLab, published on VM address `0.0.0.0:8888`.

At the end, all environment checks should show `PASS`, followed by a prominent login link:

```
★ 下一步 / NEXT STEP ★

請在你的筆電瀏覽器開啟以下完整 JupyterLab 登入連結：
OPEN THIS COMPLETE LOGIN LINK FROM YOUR LAPTOP:

  http://YOUR-VM-IP:8888/lab?token=THIS-VM-UNIQUE-TOKEN
```

If activation fails, do not install or upgrade CUDA, the driver, or Docker yourself. Do not paste a key into chat. Give the complete error output to a teaching assistant.

Step 3: Check JupyterLab status

```
docker compose \
  --project-directory ~/cudaq-course-kit \
  --env-file ~/.config/nchc-cudaq-course/course.env \
  -f ~/cudaq-course-kit/compose.yaml \
  ps
```

The service status should become `healthy`. If it is still `starting`, wait about 10–30 seconds and check again.

`PORTS` showing `0.0.0.0:8888->8888/tcp` is correct. `0.0.0.0` is the address Docker listens on inside the VM. The NCHC floating/public IP, such as `140.110.109.117`, is an external NAT mapping and will not appear in `docker compose ps`.

Step 4: Open JupyterLab directly from your laptop

`start-lab.sh` prints the complete login link for this VM. If `JUPYTER_PUBLIC_HOST` is not set, the script cross-checks the public IP through two independent HTTPS sources. It uses the result only when both sources return the same valid IPv4 address, so there is no need to configure all 40 VMs individually. If detection fails, the script falls back to a local IP and displays a warning.

```
http://140.110.109.117:8888/lab?token=THIS-VM-UNIQUE-TOKEN
```

Click or copy the link on your laptop. You do not need to enter the token separately. If the detected IP differs from the one assigned by the instructor, replace only the IP in the URL and keep the rest unchanged. TCP port 8888 must be allowed by the NCHC network rules for course participants. The complete URL contains a token; do not share it, take a screenshot of it, or paste it into chat.

Fallback: SSH tunnel (SSH port 3322)

Use a tunnel only when your laptop cannot connect directly to port 8888 on the VM.

macOS Terminal:

```
ssh -p 3322 -i ~/Downloads/bootcamp0807.pem -N \
  -L 8888:127.0.0.1:8888 ubuntu@140.110.109.117
```

Windows PowerShell:

```
ssh -p 3322 -i "$HOME\Downloads\bootcamp0807.pem" -N -L 8888:127.0.0.1:8888 ubuntu@140.110.109.117
```

It is normal for the tunnel command to display no output. Keep the window open, then use the complete localhost login link printed by `start-lab.sh`.

If the page still asks for a token, use the token printed by `start-lab.sh`, or look it up on the VM. Do not paste it into chat or a notebook:

```
grep '^JUPYTER_TOKEN=' ~/.config/nchc-cudaq-course/course.env
```

Step 5: Know the course files

The JupyterLab file browser contains:

File	Purpose
`_intro_cudaq.ipynb`	CUDA-Q fundamentals warm-up
`_intro_Ising_Calibration.ipynb`	NVIDIA Ising Calibration special unit; the Lab exposes the same value from `NVIDIA_NIM_API_KEY` as `NVIDIA_API_KEY`
`00_notebook.ipynb`	Completed 16-qubit QAOA baseline
`cudaq-doc.md`	CUDA-Q reference for the agent to read
`AGENTS.md`	Rules the agent must follow when creating notebooks
`helpers.py`	Shared data, QUBO, validation, and plotting functions; do not rewrite it
`my_code.py`	Qiskit-to-CUDA-Q migration exercise

Step 6: Complete the CUDA-Q warm-up first

1. Open `_intro_cudaq.ipynb`.
2. Run every cell from top to bottom.
3. Open `00_notebook.ipynb` and observe the difference between CPU and GPU targets.
4. Confirm the GPU in a JupyterLab Terminal:

```
nvidia-smi
python /opt/nchc-cudaq-course/smoke-test-cudaq.py
```

The smoke test checks `qpp-cpu` followed by `nvidia`. A successful run prints two `PASS` lines.

Step 7: Use OpenCode in a Terminal

In JupyterLab, select `File → New → Terminal`:

```
cd /workspace/cudaq-agentic-coding
opencode
```

In OpenCode, enter:

```
/models
```

The following models are available. `/models` should use the complete provider/model ID:

Backend	Model / `/models` ID	Credential
NCHC RAP Nemotron Super (default)	`rap-nemotron/NVIDIA-Nemotron-3-Super-120B-A12B`	`RAP_NEMOTRON_3_ULTRA_API_KEY`
NCHC RAP Nemotron Ultra	`rap-nemotron/NVIDIA-Nemotron-3-Ultra-550B-A55B`	Shares `RAP_NEMOTRON_3_ULTRA_API_KEY` with Super
NCHC RAP Gemma 26B	`rap-gemma-26b/gemma-4-26B-A4B-it`	`RAP_GEMMA_26B_API_KEY`
NCHC RAP Gemma 31B	`rap-gemma-31b/gemma-4-31B-it`	`RAP_GEMMA_31B_API_KEY`
NVIDIA hosted NIM	`nvidia-nim/nvidia/nemotron-3-ultra-550b-a55b`	`NVIDIA_NIM_API_KEY`
OpenCode Zen Nemotron	`opencode/nemotron-3-ultra-free` (Nemotron 3 Ultra Free)	Free pricing; a personal OpenCode Zen API key is still required

The course defaults to NCHC RAP Nemotron 3 Super. Super and Ultra are in the same `rap-nemotron` provider and share one RAP key; use `/models` to switch to Ultra manually. If RAP is noticeably slow or temporarily unavailable during the event, first follow the official OpenCode Zen instructions to create a personal account and API key. Then run the following in the OpenCode TUI:

```
/connect
```

Choose `OpenCode Zen` and paste the personal API key obtained from your OpenCode Zen account. Next, run `/models` and select `opencode/nemotron-3-ultra-free`. Do not paste the Zen key into chat, a notebook, shell history, or `course.env`. Connection data is stored only in the OpenCode runtime on this VM and is moved into the backup when you run `reset-manual-validation.sh`.

Zen Nemotron 3 Ultra is a limited-time free trial. It is not guaranteed to remain free, unlimited, or free of rate limits on the course date. The endpoint records trial data for safety management and product improvement. Do not submit personal data, confidential data, API keys, or Jupyter tokens. To use Zen with Jupyter AI `@OpenCode`, complete `/connect` once in a JupyterLab Terminal and then reopen Chat.

After choosing a model, first ask the agent to read the course references without creating files:

```
Read @cudaq-doc.md and @AGENTS.md, then give me a short summary of what
each one covers. Just the wrap-up. No code or files yet.
```

Then complete the GHZ exercise:

```
Write a simple script ghz.py using CUDA-Q to prepare a 20-qubit GHZ state on the GPU
("nvidia" target), sample 1000 shots, and print the counts. Refer to @cudaq-doc.md if needed,
then run it and show me the output.
```

The agent asks for permission before running shell commands or modifying files. Read and understand the command and its goal before approving it.

Optional: Enable automatic approval in Terminal

OpenCode CLI can automatically approve operations that are not explicitly denied for the current launch only:

```
cd /workspace/cudaq-agentic-coding
opencode --auto
```

You can also run one prompt non-interactively:

```
opencode run --auto "PASTE ONE STEP PROMPT HERE"
```

`--auto` skips many tool confirmations. It may allow the agent to run shell commands, modify or overwrite files, execute notebooks, access the network through the shell, or read environment variables. Back up your work first. Do not ask the agent to display API keys, the Jupyter token, or `course.env`. Stop immediately if you see an unreasonable operation. The next OpenCode CLI launch returns to individual approval unless you add `--auto` again.

Jupyter AI OpenCode Chat uses ACP. The installed version does not offer a per-chat `--auto` flag, so Chat continues to use the generated course permission configuration. Do not modify `.runtime/opencode.json` yourself. It is regenerated when the Lab restarts, and a global `allow` setting would affect every Chat. `--auto` changes only the tool approval flow; it does not override the one-step-at-a-time rule in `AGENTS.md`. Continue to paste only one course step at a time.

View today's token usage

After completing an exercise, use a JupyterLab Terminal to inspect OpenCode session, token, and tool usage from the last 24 hours:

```
cd /workspace/cudaq-agentic-coding
opencode stats --days 1
```

To include a breakdown for the five most frequently used models:

```
opencode stats --days 1 --models 5
```

Common fields:

Section / field	Meaning
`Sessions`	Number of OpenCode conversations active during the last day; this can include both CLI and Jupyter AI `@OpenCode`
`Messages`	Total messages in those sessions, including user, assistant, and tool round trips; this is not the number of prompts
`Days`	Statistics window; `--days 1` means the most recent 24 hours
`Input`	Tokens sent to the model, including prompts, conversation history, system rules, code, and tool results; repeated file reads and tool runs can increase this quickly
`Output`	Tokens generated by the model; usually much lower than `Input`
`Cache Read`	Tokens reused from the provider's prompt cache; it is `0` when the provider does not report cache data
`Cache Write`	Tokens written to the provider's prompt cache; it is `0` when the provider does not support or report it
`Avg Tokens/ Session`	Average total tokens per session; a few very long conversations can raise the average
`Median Tokens/ Session`	Median total tokens per session; usually closer to a typical conversation
`Total Cost` / `Avg Cost/Day`	Cost estimated by OpenCode from provider responses or model price data; `$0.00` for NCHC RAP does not mean that no tokens were used and is not an official activity quota or bill

Section / field	Meaning
`TOOL USAGE`	Number and percentage of calls to tools such as `read`, `bash`, `write`, and `edit`; `invalid` means the model produced an invalid or unparseable tool call and can help indicate prompt or model stability

`K` means thousand and `M` means million. For example, `29.0K` is about 29,000 tokens and `2.6M` is about 2,600,000 tokens. Tokens are not words. Chinese, English, code, and punctuation are split differently by each model tokenizer, so comparisons across models should be treated as trends only.

These statistics come from local OpenCode session data on this VM, not from an official NCHC RAP billing or quota page. After `reset-manual-validation.sh` cleans the OpenCode runtime, historical statistics and installed course skills are moved into the backup. Do not display an API key, `course.env`, or the Jupyter token just to inspect usage.

Step 8: Use `@OpenCode` in Jupyter AI

1. Open `Chat` from the JupyterLab Launcher.
2. Enter `@` and confirm that `OpenCode` appears in the list.
3. Begin with a read-only exercise:

```
@OpenCode Read 00_notebook.ipynb and explain its six code cells in beginner-friendly language.
Do not modify or execute anything yet.
```

1. Then request a plan:

```
@OpenCode Read AGENTS.md and the Step 1 prompt in README.md. Propose a plan for
01_notebook.ipynb. Do not edit files until I approve the plan.
```

1. Only allow the agent to create, run, and validate the notebook after confirming that the plan follows the rules.

Step 9: Core QAOA agentic coding exercise

Following Steps 1–4 in the repository README, create the following files in the repository root:

```
01_notebook.ipynb
02_notebook.ipynb
03_notebook.ipynb
04_notebook.ipynb
```

For every step:

1. Ask the agent to read `cudaq-doc.md`, `AGENTS.md`, the previous notebook, and `helpers.py`.
2. Review the plan first.
3. Approve notebook creation.
4. Inspect the code cells before execution.
5. Use the repository-specified 600-second external timeout to run one foreground `nbconvert`.
6. Confirm zero errors, a passing single validity assertion, and the final map visualization.
7. Do not alter sampled answers or bypass validation because a result is unfavorable.

Reference solutions are not deployed to student VMs. Instructors compare them read-only in an isolated environment only after the current step is complete.

Step 10: BYOC: Qiskit to CUDA-Q

The course image includes Qiskit, so the original program runs directly:

```
python my_code.py
```

Prompt OpenCode:

```
Read my_code.py and migrate it to CUDA-Q as grover_cudaq.py. Use @cudaq-doc.md,
run on the "nvidia" target, then run both files and explain whether the outputs match.
```

Common operations

```
cd ~/cudaq-course-kit

./start-lab.sh       # Start
./lab-logs.sh        # View live logs; Ctrl+C exits the log view without stopping the service
./stop-lab.sh        # Stop the JupyterLab container
./verify-environment.sh
```

Back up your current work:

```
cp -a ~/cudaq-agentic-coding \
  ~/cudaq-agentic-coding-backup-$(date +%Y%m%d-%H%M%S)
```

Restore a clean course repository:

```
cd ~/cudaq-course-kit
./reset-manual-validation.sh
```

`reset-manual-validation.sh` stops the Lab and moves the Jupyter/OpenCode runtime, statistics, sessions, cache, logs, OpenCode-installed `cudaq-gpu-opt-skill` / `cudaq-guide`, and acceptance log into `~/manual-validation-backups/`. It then calls `course-reset.sh` to restore a clean course repository. Root-level `SKILL.md`, notebooks 01–04, BYOC/playground programs, chats, checkpoints, `__pycache__`, and project-local agent skills move with the current repository into `~/course-backups/`. Finally, the script confirms that the new workspace exactly matches the read-only source. API keys, `course.env`, and the Jupyter token are preserved.

Troubleshooting

Symptom	Check	Action
`permission denied /var/run/docker.sock`	`groups`	Sign out and sign back in to the VM; if it still fails, ask a teaching assistant
Container cannot see the GPU	`nvidia-smi` , `docker info`	Do not reinstall the driver; provide the complete `verify-environment.sh` output
Course image not found	`docker image ls`	The base VM image was not packaged correctly; students should not pull or build it
Port 8888 is already in use	`ss -ltnp	grep 8888`	
SSH shows `Permission denied (publickey)`	Confirm the assigned IP, SSH port 3322, account `ubuntu` , and PEM path	On macOS, run `chmod 600` again; on Windows, confirm the file was not renamed to `.pem.txt`
SSH shows `Connection timed out`	Confirm the IP, network, and VPN requirements	Do not modify the VM; give the assigned IP and error screenshot to a teaching assistant
SSH shows that the host key changed	Do not ignore the warning	Ask a teaching assistant to confirm whether the IP was rebuilt or reassigned before updating `known_hosts` as instructed
Jupyter token error	Check your personal course environment	Close the old tab and sign in again with the correct token
`@OpenCode` does not appear	`opencode --version` , restart the Lab	Check `lab-logs.sh` and confirm that Jupyter AI 3.0.1 loaded normally
RAP 401/403	`/models` , confirm the activity time	Do not paste a key into chat; ask the instructor to check post-deployment injection and key status
Python package conflict	`python -m pip check`	Do not run `pip install -U` in a notebook; restore the course or ask a teaching assistant
Notebook exceeds 550 seconds	Identify the cell	Stop and report the measurement; do not force a valid-looking result

Before leaving the course

```
cd ~/cudaq-course-kit
./stop-lab.sh
```

Download or back up the notebooks you created. The instructor will revoke the activity API keys centrally. Do not take them outside the course environment.

`bootcamp0807.pem` is an activity private key. Never upload it to GitHub, put it in a public cloud link, or paste it into chat. Delete your local copy after the event as instructed.

OPEN HACKATHONS
powered by OpenACC

Ready to build with CUDA-Q

Follow the one-prompt, one-step course rhythm. Complete the warm-up first, then continue to QAOA, NCHC RAP, and Bring-Your-Own-Code exercises.

---

github.com/Squirtle007/cudaq-agentic-coding

openhackathons.org · SINICA × NCHC Bootcamp

Student Guide · 2026-07-30 · Course commit 9f399ca1 · NCHC H200 VM